CPS209 Lab 3

Custom Class: Polynomial

Preamble

In this lab you will complete one more static method, before moving on to your first custom class problem. The relevant Java code with method stubs can once again be found in the "code" directory.

Homework to Complete (Submit on D2L):

This lab requires you to do work in two Java files: LabThree.java and Polynomial.java. A separate tester is included for each of these. See below for the method descriptions for each of these files.

Reminders:

- Your methods should be SILENT. Print statements are fine for testing and debugging but remove them from your code before you submit.
- No method should take longer than a couple of seconds to pass the tester. If your code takes minutes (or *hours*, heaven help you) it means you are doing something very inefficient in your approach and/or implementation. Remedy this before submitting. Our TAs will be very busy and cannot wait 20 minutes for a single student's code to execute.

Method Descriptions – LabThree.java:

`public static String reverseVowels(String text)`

Given a text string, create and return a new string of the same length where all vowels have been reversed, and all other characters are kept as they were. For simplicity, in this problem only the characters aeiouAEIOU are considered vowels, and y is never a vowel. For example, given the text string **"Uncle Sente lives in Russia"**, this method should create and return the string **"Ancli Sunti levis en Resseu"**.

Furthermore, to make this problem more interesting and the result look more palatable, this method **must maintain the capitalization of vowels** based on the vowel character that was originally in the position that each new vowel character is moved into. For example, **"Alex Ufkes"** should become **"Elux Efkas"** and not **"elUx efkAs"**.

Hint: *You may find some handy methods in the Character utility class to help you with vowel testing and conversion…*

Method Descriptions – Polynomial.java:

Your task in this problem is to create a class Polynomial whose objects represent univariate polynomials of one variable with integer coefficients, along with the basic mathematical operations of these polynomials. As is good programming style, unless there exist good reasons to do otherwise, this class will be intentionally designed to be **immutable** so that Polynomial objects will not change their internal state after they have been constructed. Note that there is no "immutable" keyword – just don't modify the Polynomial object once it's created (in other words – no mutator methods!).

You will implement many methods for this Polynomial class over several labs, but for this lab you are only responsible for the following:

`public Polynomial(int[] coefficients)`

The **constructor** that receives as an argument the array of **coefficients** that together define the polynomial. For a polynomial of degree n, the coefficients array contains exactly $n + 1$ elements so that the coefficient of the term of order k is in the element coefficients[k]. For example, the polynomial $5x^3 - 7x + 42$ that will be used as an example in all of the following methods would be represented as the coefficient array {42,-7,0,5}.

Terms missing from inside the polynomial are represented by having a zero coefficient in that position. However, the **coefficient of the highest term of every polynomial should always be non-zero**, unless the polynomial itself is identically zero. If this constructor is given a coefficient array whose highest terms are zeroes, it should simply ignore those leading zero coefficients. For example, if given the coefficient array {-1,2,0,0,0}, the resulting polynomial would have the degree of only one, as if that coefficient array had been {-1,2} to begin with, without those pesky and redundant higher order zeros causing confusion.

As the first warning of an unexpected pitfall lurking inside this problem, the zero polynomial should be encoded as the singleton coefficient array {0} instead of the empty array that the careless removal of all leading zeros will produce in this situation.

To ensure that the Polynomial class is immutable so that no outside code can change the internal state of an object after its construction, the constructor should not merely assign the reference to the coefficients array to the private field of coefficients, but it **must create a separate but identical *defensive copy* of the argument array, and store that defensive copy instead**. This technique ensures that the stored coefficients of the polynomial do not change if some outsider later changes the contents of the shared coefficients array that was passed as the constructor argument. The JUnit test TestPolynomialOne will intentionally try to mess up some coefficient arrays later to make your code fail.

`public int getDegree()`

Returns the degree of this polynomial, that is, the exponent of its highest order term that has a nonzero coefficient. For example, the previous polynomial has degree 3. All constant polynomials, including the zero polynomial, have a degree of zero regardless of the magnitude of that constant.

`public int getCoefficient(int k)`

Returns the coefficient for the term of order *k*. For example, the term of order 3 in the previous polynomial equals 5, whereas the term of order 0 equals 42. This method should work correctly even when *k* is negative or greater than the actual degree of the polynomial, and simply return zero for such nonexistent terms.

Submission

Labs are to be completed and submitted ***individually***. Submit your completed **LabThree.java** and **Polynomial.java** on D2L, under the submission for Lab 3.

Do **<u>not</u>** archive (zip/rar/etc.) your submission. Just submit the Java files.
Do **<u>not</u>** rename your Java files.